

Polars Joins



SuperPower Polars SQL Join



ServiceDate	ServiceID	StopArrivalTime	StopDepartureTime
---	---	---	---
str	str	time	time
25-03-24	13224525	06:55:00	06:58:00
25-03-24	13224733	07:55:00	07:58:00
25-03-24	13224820	10:02:00	10:04:00
25-03-24	13224865	08:25:00	08:28:00
25-03-24	13224878	11:32:00	11:34:00
---	---	---	---
25-03-30	13265290	16:24:00	16:28:00
25-03-30	13265556	19:02:00	19:05:00
25-03-30	13266081	20:32:00	20:35:00
25-03-30	13266299	18:54:00	18:58:00
25-03-30	13266369	21:02:00	21:05:00

StopName	ServiceDate	ServiceType	StopArrivalTime	StopDepartureTime
---	---	---	---	---
str	str	str	time	time
Amsterdam Centraal	25-03-24	Eurostar	null	05:48:00
Rotterdam Centraal	25-03-24	Eurostar	06:55:00	06:58:00
Antwerpen-Centraal	25-03-24	Eurostar	07:30:00	07:33:00
Brussel-Zuid Midi	25-03-24	Eurostar	08:08:00	08:20:00
Airport Charles de Gaulle	25-03-24	Eurostar	09:38:00	09:43:00
Marne-la-Vallée-Chessy	25-03-24	Eurostar	09:53:00	null



Dataset Eurotrains

- Versión un solo archivo Parquet
- Versión particionado Hive
- Base de Datos MariaDB/MySQL

Consulta CSV en archivo local

```
pip install polars connectorx
```

```
import polars as pl
```

Definición explícita del esquema de los datos externos

Esta lectura podría parecer análoga al ejemplo de escritura en la tabla del laboratorio [anterior](#) pero entre ambas hay dos **diferencias esenciales**:

1. El esquema de los datos se define de manera explícita

2. La lectura de datos es **diferida** ya que `scan_csv()` **no se ejecuta hasta la llamada a `*collect()`* en la penúltima línea

```
# Definición de metadatos
schema = {
    "StopCode": pl.String,
    "StopName": pl.String,
    "Country": pl.String,
    "StopLat": pl.Float64,
    "StopLng": pl.Float64,
}

# Declaración de la operación
lf = pl.scan_csv(
    "Eurostar/stations.csv",
    schema=schema,
    ignore_errors=False,
    null_values=["", "NA", "null"]
)

# Ejecución
df = lf.select(["StopName", "StopCode", "StopLat", "StopLng"]).collect()
print(df)
```

StopName	StopCode	StopLat	StopLng
---	---	---	---
str	str	f64	f64
's-Hertogenbosch	HT	51.69048	5.29362
's-Hertogenbosch Oost	HTO	51.700554	5.318333
't Harde	HDE	52.409168	5.893611
Aachen Hbf	AHBF	50.7678	6.091499
Aachen West	AW	50.78036	6.070715
...	...	...	...
Praha hl.n.	PRAHA	50.085	14.441
Praha-Holesovice	PRAHOL	50.11	14.439
Baden-Baden	RBB	48.7905	8.1908
SSN Stoomdepot	RTNG	51.9363	4.5098
Aschaffenburg Hbf	NAH	49.98028	9.14361



Más Info Polars Lazy Eval

- [Polars Lazy API](#)
- [API Polars LazyFrame](#)

SQL Join con datos externos a la base de datos



WebDAV (Web-based Distributed Authoring and Versioning)

Conjunto de extensiones del protocolo HTTP que permite gestionar, editar, compartir y transferir archivos de forma remota y colaborativa desde servidores web (alojados en la web). Actúa como una unidad de red que convierte un servidor remoto en un disco duro local accesible desde cualquier lugar y sistema operativo (*Windows, MacOS, Linux, etc.*)

Más info:  [Utilizar WebDAV en Linux](#)

Join entre Parquet y CSV alojados en la Web

Q1: Horarios de salida en orden temporal creciente de los servicios *Eurostar* en cada estación la semana n (13)

```
schema = {
  "StopCode": pl.String,
  "StopName": pl.String,
  "Country": pl.String,
  "StopLat": pl.Float64,
  "StopLng": pl.Float64,
}

# WebDAV UPMdrive
stations_csv = "https://drive.upm.es/public.php/dav/files/X99My74a3LKyAK4"

# Usar scan_csv para lectura perezosa
lazy_stations = pl.scan_csv(
  stations_csv,
  schema=schema,
  ignore_errors=False,
  null_values=["", "NA", "null"]
)

# WebDAV UPMdrive
eurotrain_parquet =
"https://drive.upm.es/public.php/dav/files/XTzftsPGJMARfDj"

# Usar scan_parquet para lectura perezosa
join = (
  pl.scan_parquet(eurotrain_parquet)
    .filter((pl.col("ServiceType") == 'Eurostar') & ((pl.col("ns") == 13)) &
  (pl.col("StopDepartureTime").is_not_null()))
    .join(lazy_stations, left_on="StopCode",
  right_on="StopCode").filter(pl.col("StopName") == 'Rotterdam Centraal')
    .select(["ServiceDate", "ServiceID", "StopArrivalTime", "StopDepartureTime"])
    .sort("ServiceID", "StopArrivalTime", "StopDepartureTime")
)

# Collect (ejecutar la consulta)
res = join.collect()
print(res)
```

ServiceDate	ServiceID	StopArrivalTime	StopDepartureTime
---	---	---	---
str	str	time	time
25-03-24	13224525	06:55:00	06:58:00
25-03-24	13224733	07:55:00	07:58:00
25-03-24	13224820	10:02:00	10:04:00
25-03-24	13224865	08:25:00	08:28:00
25-03-24	13224878	11:32:00	11:34:00
...	...	...	...
25-03-30	13265290	16:24:00	16:28:00
25-03-30	13265556	19:02:00	19:05:00
25-03-30	13266081	20:32:00	20:35:00
25-03-30	13266299	18:54:00	18:58:00
25-03-30	13266369	21:02:00	21:05:00

Join entre tabla *MariaDB* y CSV alojado en la Web

Q2: Itinerario (secuencia de estaciones en orden temporal creciente) de un servicio concreto (13224525)

```

schema = {
    "StopCode": pl.String,
    "StopName": pl.String,
    "Country": pl.String,
    "StopLat": pl.Float64,
    "StopLng": pl.Float64,
}

# URL archivo. Lee directamente desde la Web
stations_csv = "https://drive.upm.es/public.php/dav/files/X99My74a3LKyAK4"

# Usar scan_csv para lectura perezosa
lazy_stations = pl.scan_csv(
    stations_csv,
    schema=schema,
    ignore_errors=False,
    null_values=["", "NA", "null"]
)

# Conexión con la base de datos
uri = "mysql://eurostar:eurostar@mariadb:3306/eurostar"
sql = """
    SELECT tts, StopCode, ServiceDate, ServiceType, ServiceID,
    StopArrivalTime, StopDepartureTime
    FROM eurostar.EuroTrain
    """

# Usar read_database_uri para acceder a la tabla
lazy_join = (
    pl.read_database_uri(query=sql, uri=uri).lazy()
    .filter(pl.col("ServiceID") == '13224525')

```

```

.join(lazy_stations, left_on="StopCode", right_on="StopCode")
.sort("tts")

.select(["StopName", "ServiceDate", "ServiceType", "StopArrivalTime", "StopDepartu
reTime"])
)

# Invocar collect provoca la ejecución de lazy_stations y lazy_join
res = lazy_join.collect()
print(res)

```

StopName StopDepartureTime	ServiceDate	ServiceType	StopArrivalTime	
---	---	---	---	---
str	str	str	time	
time				
Amsterdam Centraal 05:48:00	25-03-24	Eurostar	null	
Rotterdam Centraal 06:58:00	25-03-24	Eurostar	06:55:00	
Antwerpen-Centraal 07:33:00	25-03-24	Eurostar	07:30:00	
Brussel-Zuid Midi 08:20:00	25-03-24	Eurostar	08:08:00	
Airport Charles de Gaulle 09:43:00	25-03-24	Eurostar	09:38:00	
Marne-la-Vallée-Chessy null	25-03-24	Eurostar	09:53:00	